This tree contains all the shortest paths from the source vertex to the other vertices; but… the output of the BFS algorithm is just a dictionary, how can we reconstruct these paths from the `parents` container that is returned?

Listing 14.3 describes the algorithm: it starts from the goal vertex (the last vertex in the path), and reconstructs the path backwards, at each step looking for the parent of current vertex. In simple graphs, since there is only one edge between an ordered pair of vertices, if we know that we moved from vertex v to vertex u, the edge traversed becomes implicit; in multigraphs, we would have to keep track of the actual edge chosen at each step.

The code in listing 14.3 assumes that the `parents` dictionary is well formed, that the graph is connected and that there is a path from source to destination: if all of that holds, there are only two cases where current vertex can have a `null` parent: either `current` is the destination vertex (and in that case, it means it wasn't reachable from the source), or `current` is the source vertex.

If `parents[destination] != null`, in fact, this means that the destination was reached by traversing a path from the source (because of the way BFS works), and it can be proved by induction that there must be a chain of vertices between those two vertices.

Let's see the algorithm in action on the result of the run shown in figure 14.10: `bfs`, on that graph, using v_1 as source, returned the following values for parents: